

ROAD SEGMENTATION FROM AREAL IMAGERY

Bessö Erik, Berger Colin, Zraggen Jan

Abstract—In this project, we evaluate the performance of various convolutional neural network (CNN) based architectures for road segmentation from aerial imagery, focusing on U-Net and ResNet models. Our results show that the U-Net outperforms both ResNet-18 and ResNet-36 in terms of F1-score and requires fewer epochs to converge. Notably, a medium-sized UNet achieves better performance than a smaller and larger version. We also explore the impact of different image transformations and patch sizes, finding that larger patch sizes and certain transformations, such as flipping and erasing, enhance model performance. Our best performing model, a fine-tuned pre-trained MobileNetV3, achieved an F1-score of 0.880 converging in just a few epochs. These findings underline the importance of architecture selection, training duration, and data augmentation strategies for optimizing road segmentation tasks.

I. INTRODUCTION

Image segmentation is a fundamental task in computer vision and has applications across multiple business domains. With the rise of AI, more powerful computational resources become available thus resulting in the ability to train models with more parameters.

In this report, we assess the performance of different models for road segmentation from aerial imagery. The models are characterized by their size, that is the number of learnable parameters. In the report, we mainly work with convolutional neural networks (CNN), specifically the ResNet and U-Net architectures. Furthermore, we evaluate the impact of applying image transformations in training and tuning hyperparameters. We also attempt to train an optimal model by fine-tuning a pre-trained segmentation model.

Lastly, we give a brief discussion of the ethical risks that arise from our training methodology and the model's applications.

II. METHODS

A. Template and Libraries

For implementing the machine learning pipeline, we use the LauzHack template ¹. PyTorch ² is used as the machine learning framework and Hydra ³ to handle different training configurations. We use Tensorboard ⁴ and Matplotlib ⁵ to log training and testing results and generate plots.

¹LauzHack template

²PyTorch

³Hydra

⁴Tensorboard

⁵Matplotlib

B. Feature Engineering

1) Defining Patch Sizes

The objective is to classify 16×16 pixel patches to determine whether they represent road regions or not. A straightforward approach involves cutting the input images into grids of 16×16 patches. Then, during training and evaluation, we give the model one patch and predict a single value (1 if it is a road region and 0 otherwise). However, incorporating larger contextual information around each patch has the potential to improve classification performance. To explore the impact of context, we experiment with varying patch sizes: 16×16, 32×32, 128×128, and 400×400 (the full train image). The model now not only predicts whether a single region, but for multiple regions, e.g. for the 32×32 patch size the model predicts 2x2 regions. To assess the models' performance across different context sizes, we train a simple U-Net.

2) Image Transforms

Before training the model, we apply different transforms to the images. The reason for this is to make the model more robust and avoid overfitting, as it means the training dataset consists of more varied data.

We apply the transforms batch-wise meaning we apply the same transform on the whole batch. Also, we apply the transforms on the fly, meaning we do not augment the size of our dataset in a preprocessing step. Therefore, our dataset does not grow in size and can fit entirely into the RAM or GPU. We apply the following transforms: scaling, normalization, random vertical flipping, random horizontal flipping, Gaussian noise, and random erasing.

Scaling transforms the RGB values from [0, 255] to an interval between [0, 1]. This is the first transformation we apply, as the normalization and Gaussian noise would not be meaningful otherwise. For normalization, we use the mean and standard deviations from ImageNET⁶. We apply vertical and horizontal flips independently at random with a probability of 50%. Note that we also flip the mask accordingly, so it correctly reflects the roads of the transformed image. We add Gaussian noise with 0 as mean and 0.1 as standard deviation. Furthermore, we randomly erase parts of the image. We think this would train our model to be invariant to covered roads (e.g. trees or vehicles are covering the road).

⁶ImageNet dataset

By default, we use scale and normalize the images. Then, to assess the effect of the other randomized transforms, we train a simple U-Net on the full image, with each transform applied individually. At the end, we try to combine multiple transformations and train a simple U-Net.

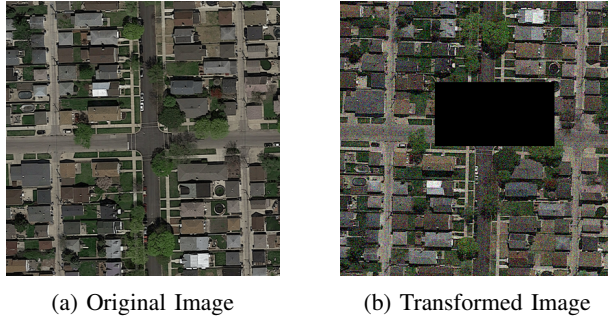


Fig. 1: Adding Gaussian noise, flipping and erasing.

C. Models

In order to find the best model, we test several different model architectures of varying sizes. To determine the size of the model, we calculated the number of trainable parameters.

1) Baseline

For the baseline model we use three fully connected layers with 512 nodes as the hidden layers. ReLU is used as the activation functions as it allows modeling non-linear behavior.

2) ResNet

In the experiments, we try different versions of ResNet architectures [1]. Specifically, we train ResNet models of depth 18 and 36 commonly referred to as Resnet18 and ResNet36. Furthermore, we use a dropout rate of $p = 0.3$.

As the ResNet attempts to predict the class for each individual pixel, we use one fully connected layer to transform the model output to predict patches of size 16×16 instead.

3) UNet

A popular architecture for road segmentation from aerial imagery are adaptations of the UNet [4], a UNet consists of an encoder pathway, for which we are using 2×2 max pooling and ReLU [5] that captures context and a decoder pathway, that enables precise localization, this can be adapted for the task as proposed by Khan et al. [3]. In order to predict the 16×16 pixel patches, the UNet is appended with additional convolution layers with stride, followed by flattening, in order to adapt for patch predicting. We implemented the UNet with different sizes determined by the number of channels

per level, we test a small (4/8/12/16 channels), a medium (16/24/36/48 channels), as well as a large model (32/64/128/256 channels).

4) Adapted DeepLabV3 (pretrained)

Additionally, we try using a pre-trained image segmentation model, as it has potentially already learned features that capture the semantics of our task. In order to do so, we used DeepLab's pre-trained MobileNetV3 [2], as it shows good performance on segmentation tasks and is of reasonable size.

D. Training

The dataset consists of 100 images of size 400×400 . We split the dataset into one training set of 90 images and one validation set of 10 images. For training, we use the Adam optimizer, a learning rate scheduler and the Binary cross entropy as the loss function.

To test the patch sizes and transforms, we use 50 epochs and 300 iterations per epoch. To assess the performance of the UNet and ResNet architecture, we use 30 epochs and 100 iterations per epoch.

The model is evaluated on the validation set between each epoch. Where we compare Inference results on the training and the validation set to assess performance and overfitting. We store the best model based on the F1-score achieved on the validation.

We conduct the experiments and the training of the final model on the Izar cluster⁷.

III. EXPERIMENTS

The first experiment evaluates the U-Net performance on the different patch sizes listed under methods. The second experiment evaluates the effects of applying different transformations on the images. We test applying the baseline transformation with either random vertical/horizontal flipping, Gaussian noise or random erasing. Then, we combine the baseline transformation with flipping and erasing. Lastly, we test applying the baseline transformation with all the transformations mentioned above (we call this transformation noisy). To evaluate the effect of the transformations applied we use the small UNet model with the optimal patch size found in the first experiment.

Lastly, we test different model architectures on the optimal patch size and transformation found in the previous experiments. Also, we fine-tune the pre-trained model.

IV. RESULTS

A. Feature Engineering

Training the small version of the U-Net on different patch sizes, we observe an increase on F1-score when increasing the patch size. This can be seen in figure 2.

⁷Izar cluster

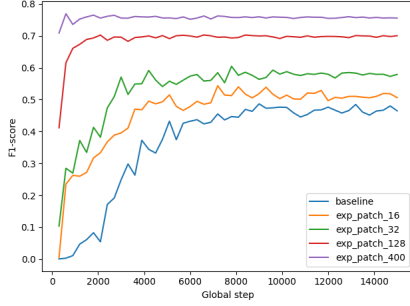


Fig. 2: Effect of different patch sizes.

Therefore, we train the transform experiment on the small U-Net with the full image. We observe that all transforms improve compared to the baseline transform (exp_patch_400). This can be seen in figure 3. When applying a single transform, flipping had the biggest impact, followed by erase and noise. We observe that the combination of flipping and erasing achieves the best overall performance, while the noisy transform (all combined) performs worse than just flipping.

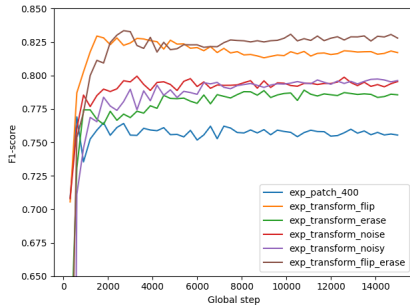


Fig. 3: Effect of different image transformations.

B. Models

We observe that the U-Net architecture outperforms both ResNet-18 and ResNet-36 architectures in terms of performance and requires fewer epochs to effectively learn the underlying information. In contrast, the larger ResNet model demonstrates a noticeable latency in learning but appears to still be improving when training is stopped, suggesting it could benefit from additional epochs. For U-Net, the medium-sized model, defined by the number of channels used, achieves superior performance compared to both its smaller and larger counterparts, while for ResNet, the larger architecture performs better overall. This can be seen in figure 4.

Table I summarizes the best F1-scores on the validation set for each of our experiments. We can observe that our best performing model is the fine-tuned pre-trained model with an F1-score of 0.880.

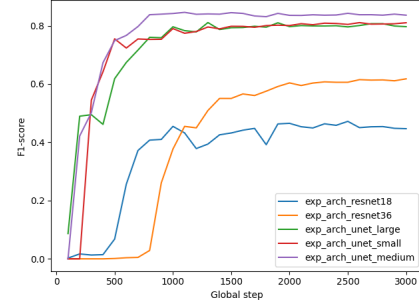


Fig. 4: Effect of different model architectures.

Experiment	F1 score
Baseline	0.487
Patch size 16	0.544
Patch size 32	0.604
Patch size 128	0.703
Patch size 400	0.769
Transform erase	0.789
Transform noise	0.800
Transform flip	0.830
Transform flip erase	0.834
Transform noisy	0.797
Architecture ResNet18	0.472
Architecture ResNet36	0.618
Architecture UNet small	0.811
Architecture UNet medium	0.846
Architecture UNet large	0.811
Fine-tuned MobileNetV3	0.880

TABLE I: Results of all our experiments.

In figure 5 we show the predictions by our best model on an image of the validation set. In green, we show the ground truth and the red boxes are the predictions of our model.

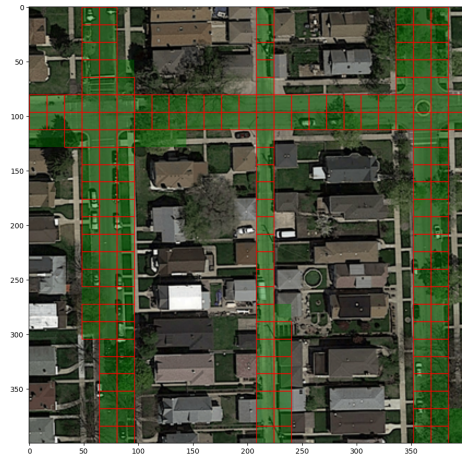


Fig. 5: Prediction example MobileNetV3

V. DISCUSSION

A. Feature Engineering

Our experiments demonstrate that context is crucial for learning effective representations. The U-Net architecture excels at capturing local information from context [3], which explains why using the largest possible patch size yields the best performance. Interestingly, adding noise on top of random flipping and random erasing, did not improve the performance. This may be attributed to the homogeneity of our dataset, as it predominantly consists of images from similar neighborhoods and of similar quality. Consequently, adding noise does not improve the model on the validation set, as the validation images are not noisy.

B. Models

The U-Net model outperforms ResNet-18 and ResNet-36 in terms of F1-score, requiring fewer epochs to converge and effectively learning the data patterns, highlighting its efficiency in training. This aligns with existing literature, where U-Net has been widely successful in image segmentation tasks [3][5]. In contrast, ResNet-36 shows slower initial learning but continues to improve after training ends, suggesting that larger models may require more time to fully leverage their capacity. Interestingly, the medium-sized U-Net outperforms both its smaller and larger counterparts, indicating that additional layers may lead to overfitting rather than improved feature learning. For ResNet, the larger architecture, despite its slower convergence, ultimately yields better results, suggesting that a larger size becomes beneficial with sufficient training time.

VI. ETHICAL RISK ASSESSMENT

A. Bias towards urban neighborhoods

The dataset used for training the segmentation model poses some ethical risks. The dataset contains exclusively satellite images of a certain type of urban area, see image 6. This means our model is only exposed to asphalt roads and may therefore overfit and perform worse on other types of road such as dirt roads.

The consequence of this is a worse performance in rural areas and developing areas as dirt roads are more common there. Furthermore, in a hypothetical scenario where the model is used in the context of road maintenance it could mean rural and developing areas receive worse maintenance than richer areas. Further, certain demographics are overly represented in rural and developing areas and this would mean that these demographics would receive worse maintenance.

To reduce the risk of overfitting to a certain type of area and roads we apply Gaussian noise to distort the roads and thus increasing the robustness of the model to

other types of roads. In addition, we considered using an external dataset containing other type of areas. However, due to the scope of the project this approach was not further examined.



Fig. 6: Sample satellite image from the dataset

B. Application for military purposes

Road segmentation has application in several different domains, one of them being in the military. We hypothesize that the model may be used in military applications such as planning air strikes on roads cutting off essential supplies for local population.

In this scenario misclassification comes with high consequences. For example, misclassifying a house as a road would risk people's life and thus it raises the ethical question about using computer vision in these scenarios.

Regarding this ethical risk we did not take any measures to reduce it as it would impact the performance of our model. Furthermore, we evaluate this risk to be very low as better models for this purpose exists.

VII. CONCLUSION

This project shows that a medium-sized UNet architecture achieves a very good F1-score on the road segmentation task. Training on the whole image, i.e., adding as much context as possible, results in better performing models. Furthermore, adding image transformations such as random horizontal and vertical flipping as well as random erasing, lead to more robust models and ultimately higher F1-scores. Finally, we find that using a pre-trained segmentation model and fine-tuning it on our dataset, achieves slightly better performance than the models we fully train our self. Notably, the pre-trained model achieves a F1-score of 0.880 on the validation set in just a few epochs. On AICrowd our model achieves a score of 0.862.

REFERENCES

- [1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *arXiv preprint arXiv:1512.03385*, 2015.
- [2] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1314–1324, 2019.
- [3] Mohd Jawed Khan and Pankaj Pratap Singh. Advanced road extraction using cnn-based u-net model and satellite imagery. *Heliyon*, 9(1):e12345, 2023.
- [4] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. *arXiv preprint arXiv:1505.04597*, 2015.
- [5] Douglas Stumpp. Extracting road networks from satellite images with neural networks to contribute to risk mitigation, 2023. Machine Learning Project Report.